期末复习：Week1~2：kernel & dual mode operation (kernel-user)

processes

memory

Files and I/O

interface of OS: shell、system call

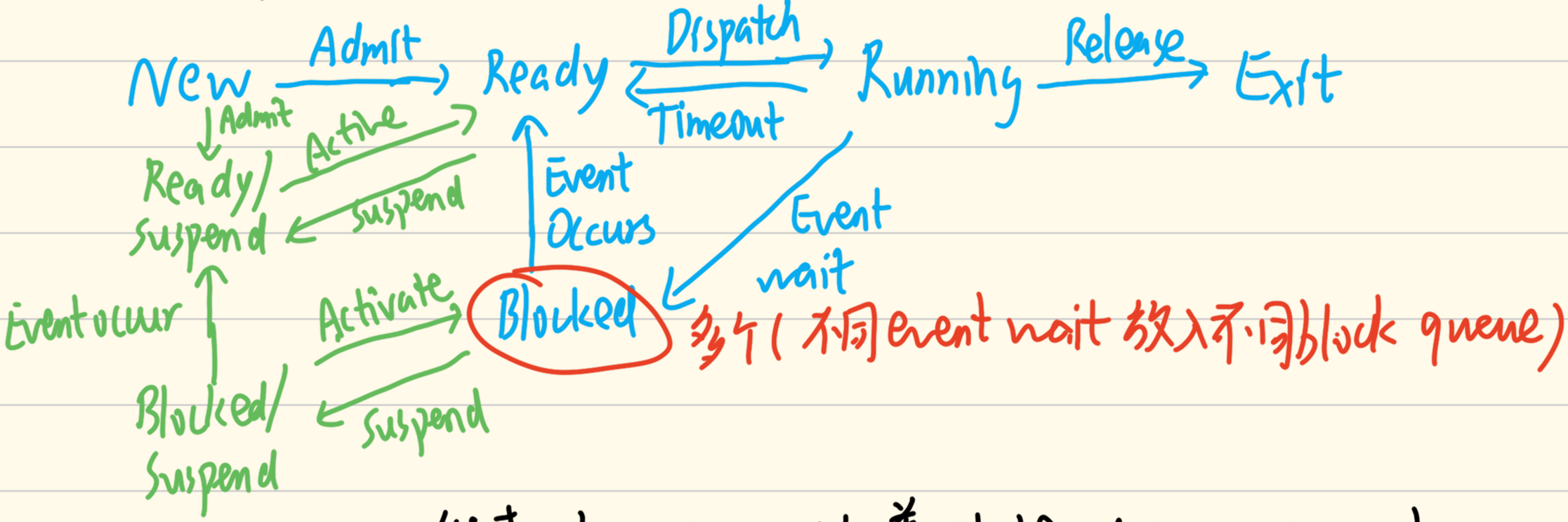
Week3~4 concurrency、threads

deadlock

week 1~2 process, process state, protection (dual model operation), context switch
process schedule

process: an instance of a running program with current states and resource

5-state process model (change process state)



Kernel 中 PCB 代表1个 process (存着 state, data, resource)

dual mode operation:

User → Kernel : set system mode + save user PC

kernel → user : clear system mode + restore user PC

Mechanism	Cause	Use
Interrupt	External to the execution of the current instruction	Reaction to an asynchronous external event
Trap	Associated with the execution of the current instruction	Handling of an error or an exception condition
Kernel call	Explicit request	Call to an operating system function

3 operation: system call : process request system servile 通过 call gate 得到使用服务

interrupt : 外部 event trigger context switch

trap/exception 内部 event trigger context switch

发生这3个 operation 时 process switch

process switch 过程: save data/process state → update PCB → move process to queue

↓

select new process → update its PCB → update memory

↓

restore context of 选中的 process

process schedule: ① 怎么 choose ② when switch 另一process
③ which process to choose next

↓

目的：缩短 response time、给定时间最大化完成process、use resource efficiently
阻止 starvation

process type: batch process CPU密集型，主要 data process, calculation
interactive process I/O密集型，主要 interact with external device

schedule algorithm:

FCFS (FIFO)：按request顺序分配CPU
优：simple to implement，若all process同长度效果好
缺：short process 会 wait 长时间

SJF：先给short process分配CPU，再给long process分配CPU
优：average wait time 短
缺：导致 starvation

non-preemptive 进程完成不被打断

会算每个算法 average wait time
会画 Gantt chart

preemptive → RR (with q=...): 每个process运行1个time slice后被preempted 顺序是FCFS

优：缩短average wait time，更fair

缺：导致starvation

priority. process分配priority value，scheduler先选high priority process，再选queueing' low priority process. (不同priority放入不同ready queue)

Multi-Level Feedback Queue: 从top level priority process开始run

process被preempted就go down a level

被I/O block就 go up a level

根据不同priority queue改变preemption time

process control

process can create a process process的unique identity是PID

os.fork() 可创建process的copy with new PID. new process是原process的child，一开始共享整个地址空间，但后续可独立修改操作

process control with python

- The **os** module contains functions that call POSIX system calls:
  - **os.fork()** – system call to create a copy of the current process, and start it running
  - **os.execv (various versions)** – system calls to *change the program* being run by the current process 改变当前process正在执行的program
  - **os.waitpid()** – system call to wait for a process to finish parent被block等child执行完
  - **signal.signal()** – system call to send a notification to another process (from **signal** module) 为signal注册信号处理函数，信号来时执行→向process发通知

scheduler priority in Linux

niceness $\in [-20, 19]$

least nice, superuser only (自己的行) → nicest, 其它process go first

completely fair scheduler (CFS):

File system：file、directory，file allocation，相关linux命令

定义：layer of OS，将disk的block interface 转换成file、directory

功能：Disk management：将disk block 整理成file

Naming：interface 按名称找file

protection：保证data secure

durability：保持file durable

file：abstraction for non-volatile storage

Contain：data + metadata

requirement：可变大小、保护、并发、管理free disk block

directory：the name of file is stored in a directory, which is itself a file

open file过程：① open()系统调用从disk读取inode，存入system-wide open file table

② 1个entry加入per-process open file table且其中1个index通过open()返回给我

close file过程 ①释放对应per-process table entry ②memory cache中所有data写入disk

PHP中的file/directory operation (命令在filesystem15~17页)

file share: file被多user访问

设权限 owner (u) read (4)
group (g) write (2)
world (o) execute (1)
→ chmod 744 filename.txt

file link: 将不同文件名与同一文件关联(用户不同filename访问同一file)

① hard link: 通过在directory中create entry指向文件inode

ln filename link: 一个file可有多个hard link, 只有删除所有hard link才真正删除file

hard link不允许link directory

② soft link: 其中包含一个file/directory的full path name, 类似于windows shortcut

ln -s filename link: 当access soft link时OS根据其中path找目标file/directory

若file/directory被移动/删除, soft link失效

inode: index node. that store metadata about the file it refer to

metadata → file size, location, owner/group, permission

↓

存在 disk block groups 中更接近 data 可用 ls -i (ls -lai) 查

↓

目录包括 { a list of filename
pointer to inode

file allocation

inode structure 是1种追踪 which disk block 对应 which file 的方法

其包括、contiguous 连续分配 优：easy implementation、good performance
缺：external fragmentation、poor scalability

linked list file由a list of node组成，每个node包含data和指向next node的pointer
优：可动态分配内存，内存利用率高，无internel fragment
缺：slow to random access、指针有开销

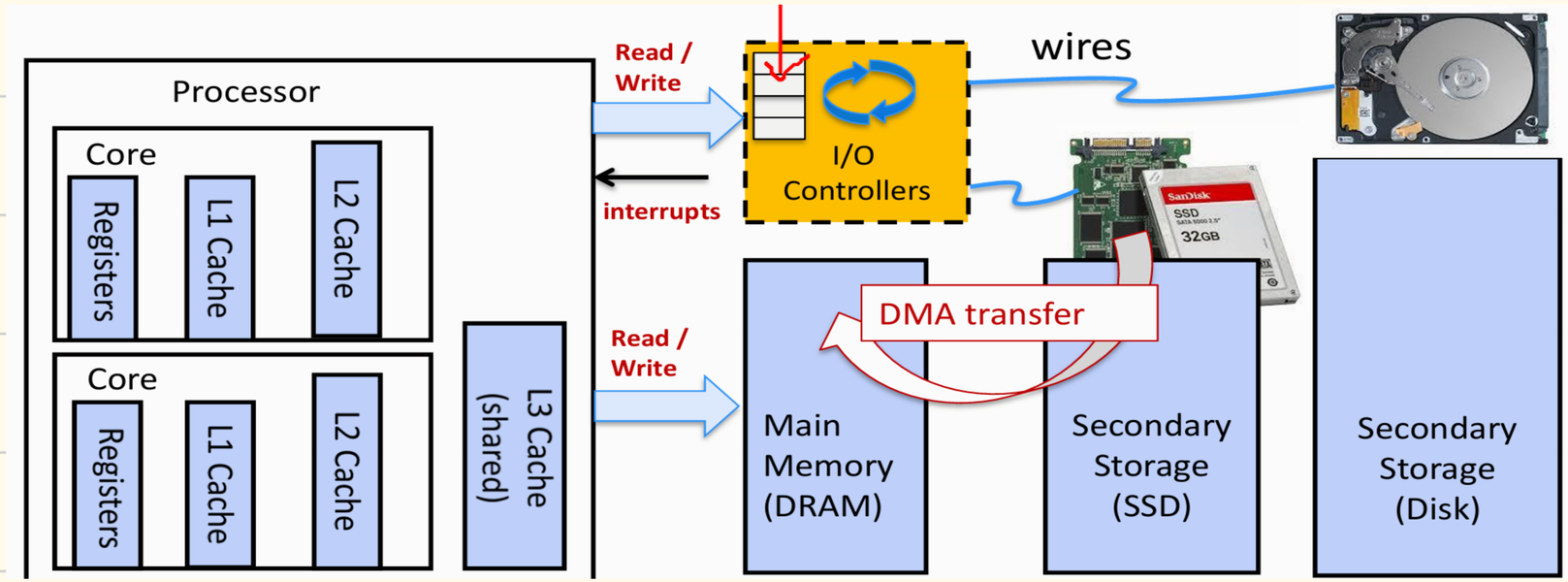
file allocation table FAT 通过 file allocation table 记录 file 在 disk 上位置

将 disk 划分为 cluster, file is a collection of cluster 通FAT查找存储file

优：simple to implement

缺：slow to find block, internal fragment

Disk and I/O 的写入读取、driver、DMA, disk(HDD、SSD)



bus: a set of wires 用于device间通信 + data transmission的协议

Device Controller

① external connection 遵循 universal bus standard.

② 是 interaction between OS and I/O device interface

任务: read/write operation

执行 validation / error correction

communicate with CPU

OS与I/O交互过程：

① OS用status register看device是否busy

② OS写入data register并发command register（同时status register为BUSY）

③ controller读data register、command register并执行命令

④ OS通过status register看何时命令执行完 —— 成功执行：clear command + reset BUSY status
出错 status register → ERROR

CPU如何与外部设备交流 —— memory mapped I/O

CPU用load/store指令与外部设备交互，好像device是内存的一部分

对external device的读写是通过对memory中特定位置进行读写实现，这些特定位置是用来存external device的buffer和register

将外部设备的register/buffer映射到memory上使CPU易访问并操作

但这部分内存（CPU与设备交互时）不能用于其它操作

1) 如何检查device完成I/O操作

① interrupt：device在need service时产生interrupt

优：处理unpredictable event好

缺：high overhead

② polling：1) 定期检查device的status register

优：low overhead

缺：waste many cycle on polling

2) 都结合来完成

CPU与device间如何data transmission

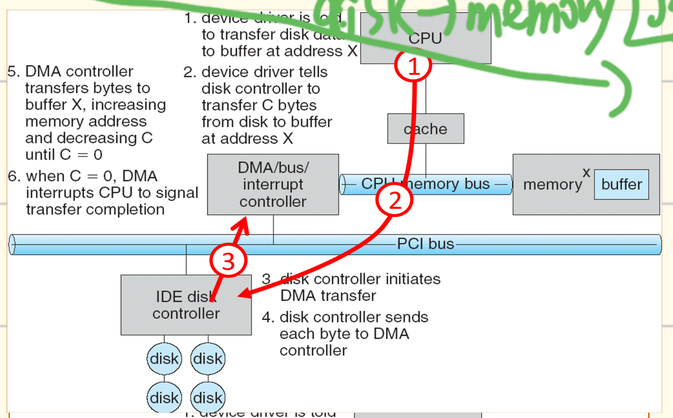
① programmed I/O：each byte transmission 通过CPU的instruction

优：simple hardware，easy to program

缺：consume CPU时间与data传输完成正比、CPU长时间占用 → efficiency↓

② direct Memory Access (DMA)：device controller 直接访问 memory bus（绕过CPU）

DMA controller 可根据CPU指令直接将 data 设备⇄内存，不用CPU处理



disk→memory 过程

① device driver 被告知将 data 从 disk 传到 buffer（地址X）

② driver 通知 disk controller 从 disk 传 C byte data 到 buffer

③ disk controller 启动 DMA 传输

④ disk controller 将 1 byte data 发给 DMA controller

⑤ DMA controller 将 1 byte data 传到 buffer，递减 C

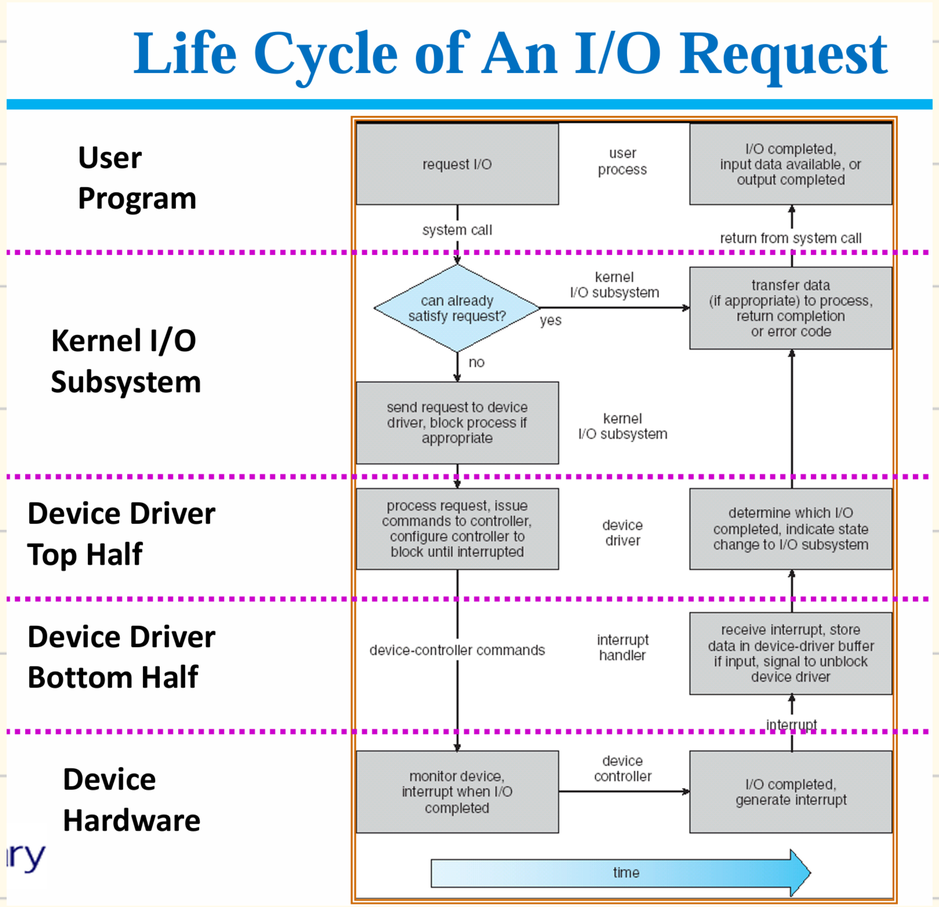
⑥ C=0 时（传完），DMA 中断 CPU 表示传完了

CPU 仅在传输的 stand + end 时被中断

- synchronous, blocking : CPU中断后一直等I/O就绪再执行(期间一直wait)
- asynchronous, non-blocking : CPU中断后其它process执行, I/O就绪返回上层进程

device driver : device-specific code in kernel that interact with device

- top-half : 从 system call 中 access in call path
- bottom-half 作为 interrupt routine 运行

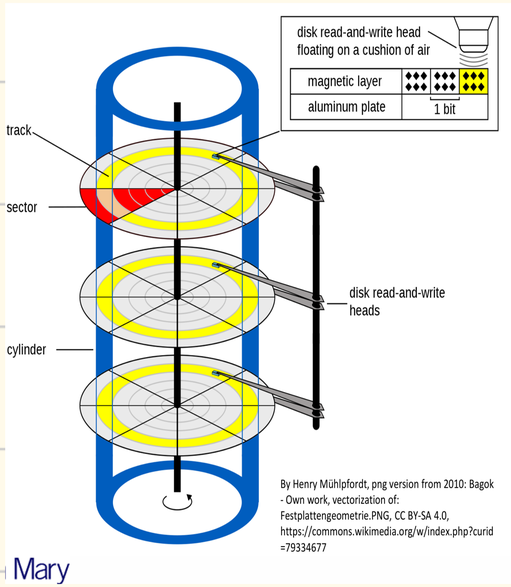


Hard disk drive (HDD) 一圈圈的轨道

track : ring around disk surface

cylinder : stack of tracks that are the same distance from spindle 一组track构成的截面

sector : segment of track track上一部分



寻址方法

CHS (cylinder-head-sector): 地址由柱面号、磁头号、扇区号组成

LBA (logical block addressing): 不考虑 physical structure，将disk上的有单元从0编号

OS访问disk数据时只提供逻辑块地址，控制器根据物理参数转换为CHS地址来读写

Disk scheduling: ① FIFO：按request顺序seek

② SSTF：每次seek最近address

③ scan：根据初始direction一直seek，到头再反方向seek

④ C-scan：根据初始direction一直seek，到头再从另一端开始seek（同方向）

SSD (solid-state drive) 用集成电路持久存data不涉及移动机械部件

a device that store data persistently using integrated circuits without any involvement of moving mechanical parts

每个chip由bank组成，bank分成blocks(256KB)，block分成page(4KB)

read：快(controller定位对应page发电信号来读)

write：写前先erase原有的block再发电信号写入

erase：以block为单位擦，慢，且对寿命有影响

随机访问功

顺序访问功

写入

可靠性

寿命

HDD	SSD
因磁头移动导致显著延迟，性能衰减	具有显著优势，无磁头移动造成的延迟
头寻道减少，性能差异较小	读取性能从相似到快几个数量级
写入性能受盘片状态、磁头移动等影响，差异较大	写入性能受写入磨损、容量等因素影响，差异较大
机械部件易故障，有 “死亡之音”	无机械部件，更可靠
平均失败率 6 年，预期寿命 9 - 11 年	寿命取决于 Drive Writes Per Day（DWPD），一般寿命长

week3 memory management

重点

要解决问题：① relocation　② efficient memory use

page　　TLB

solution：

virtual memory

locality and page fault(局部性+页面错误)

page replacement

内存管理的requirement

relocation: move the program from 1 place to 另1 place的能力

变量：reference to memory location

protection & sharing
program访问不允许的memory导致trap
但有时需多个process访问相同memory

每个process work in its own address space

logical：reference to a memory location that独立于当前使用的内存

logical address space: the set of address available to program

physical：actual location in memory 总小于logical address space

address translation: logical → physical

relocation 的 old solution: partition memory

- fixed partitioning：把memory分成固定"块"，分区不变 → internal fragmentation
- dynamic partitioning 加载program时创建分区 → external fragmentation

physical address = logical address + base register

现在的solution —— memory pages (divide memory into equal-size pages)

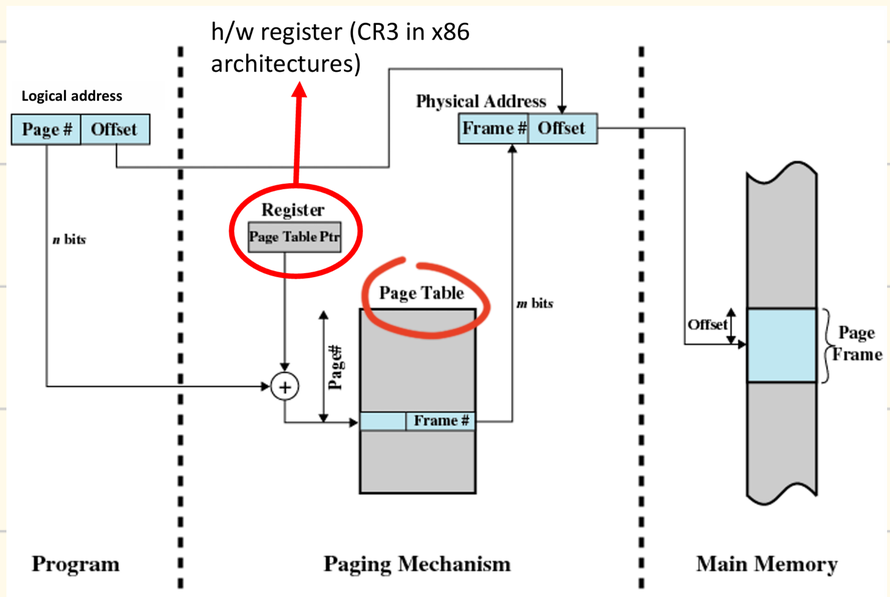
对于physical address: fixed-size memory frames (discontinuous)

logical address: process有 memory pages (与frame相同大小)

优: 无external fragmentation + 易implement + 易protection/sharing + 易分配physical memory

要给each process分配page table (每个process仅可访问被分配到的frame)

logical address (4B) { page number 20bit · 有2^{20}个page
offset 12bit = 1个frame.size=4KB



对于32bit的logical address, 每个bit可指向1个byte

$2^{32}=2^{20}+2^{12}$ → page size=4KB + 有2^{20}个page

page table是将logical → physical的结构 4MB

已知1个PTE=4B, 1个page size=4KB 则用来映射

这些page的page table大小=$2^{20}\times 4B$=4MB

Exercise 1: Memory Translation

Part of the page table for a process **P** is given below. Assume that process **P** is the currently running process. All numbers given are decimal and start from zero. All addresses are memory byte addresses. You can assume that the page size is **4096** bytes. **Note**: No translation from decimal to binary is needed.

Logical Page Number	Present (or Valid) bit (1=Present)	Frame Number
0	1	10
1	1	11
2	0	-
3	1	3
4	0	-
5	1	1

What physical address (in decimal), if any, does each of the following logical addresses correspond to? Explain how you arrive at the answer.

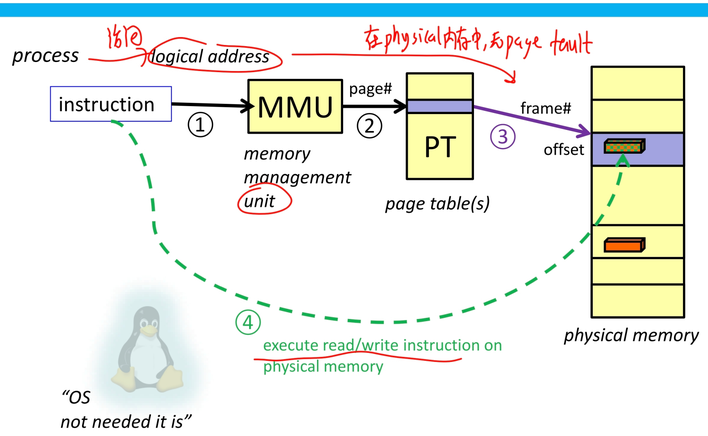
- 307
- 12728
- 8192

Queen Mary University of London

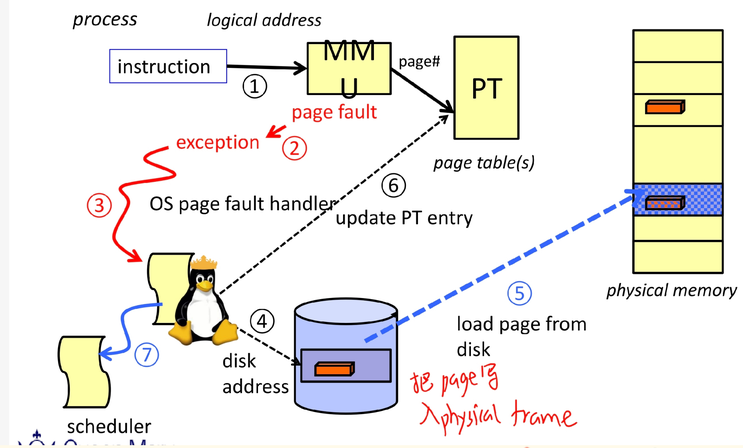
virtual memory:

程序不需all pages (有些功能少用), 仅加载部分进程好处是程序可be written for a larger address space

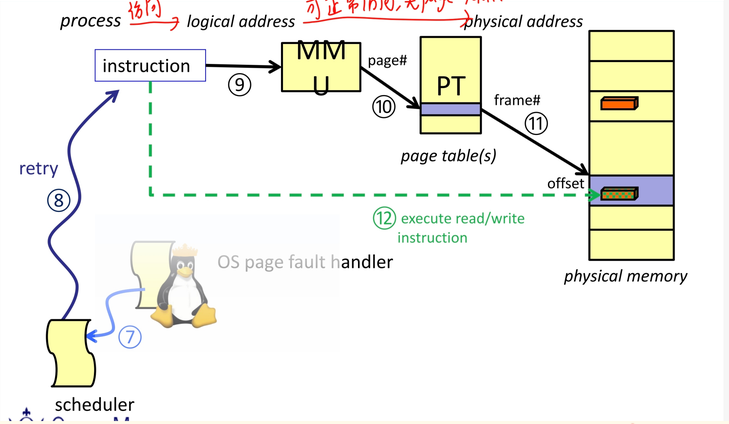
Case 1: No page fault (as see before)



Case 2: Page fault Handler (II)



Case 2: After PF Handling (III)



若data存在memory中, 访问logical address可通MMU找到physical address

若data不在memory中, 访问logical address有page fault(exception) OS转为Kernel把page写入physical address

此时再访问logical address无page fault

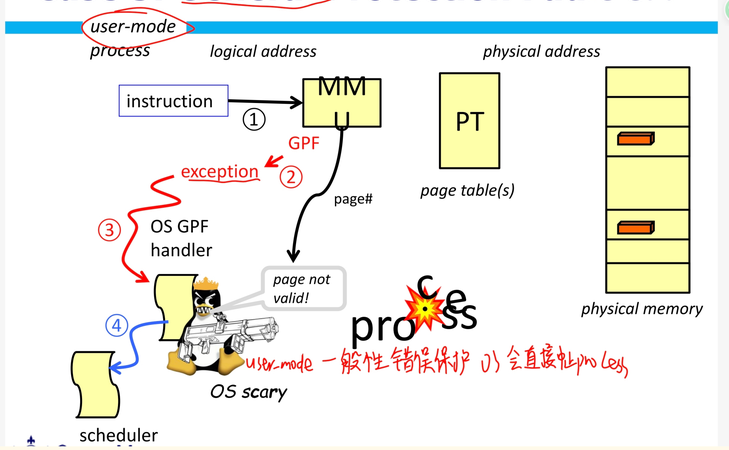
若 page that process need is not in memory ⟹ page fault

page fault导致 context switch

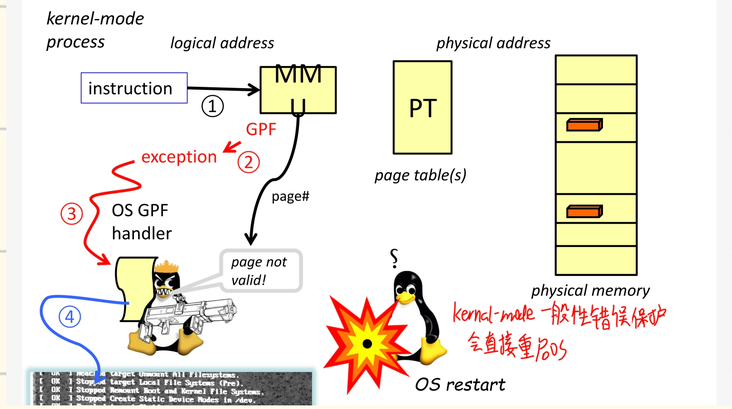
对于 general protection fault(GPF) ⟶ page fault是特殊的

in user-mode 重启process　　in kernel mode 重启OS

Case 3: General Protection Fault GPF



Case 4: GPF in Kernel-Mode

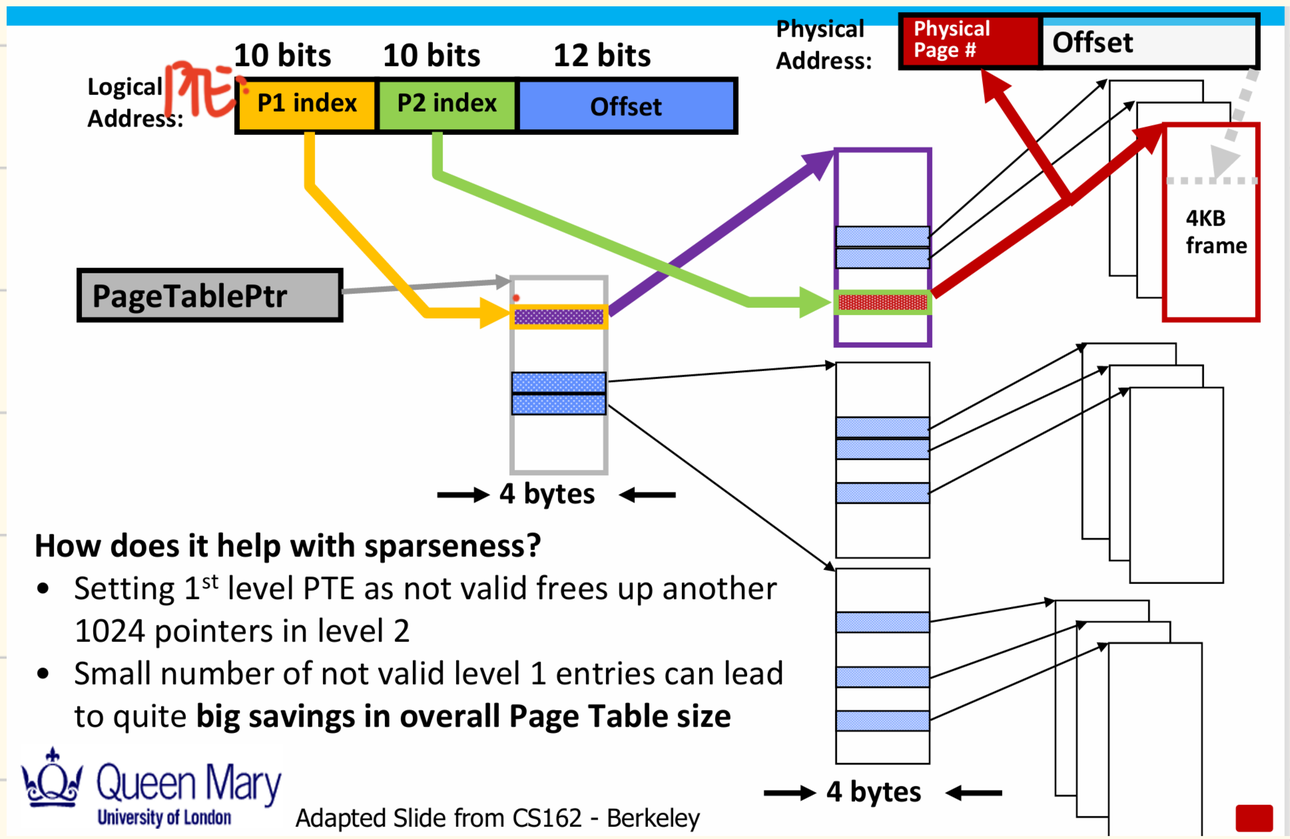


如何解决page table size 和 sparsity of logical address?

问题：若logical address大且page size小 → page table会很大(page多) → 占大量内存

process不会用全部logical address space，page table is sparse → waste space

solution: multi-level page table (1个pointer指1 level of indirection，更多pointer指向其它level)



Level 1 page table = 2^{10} PTEs

和 each PTE = 10+10+12 = 32bit = 4B

Level 2 page table size = $2^{10} \cdot 4B = 4KB$

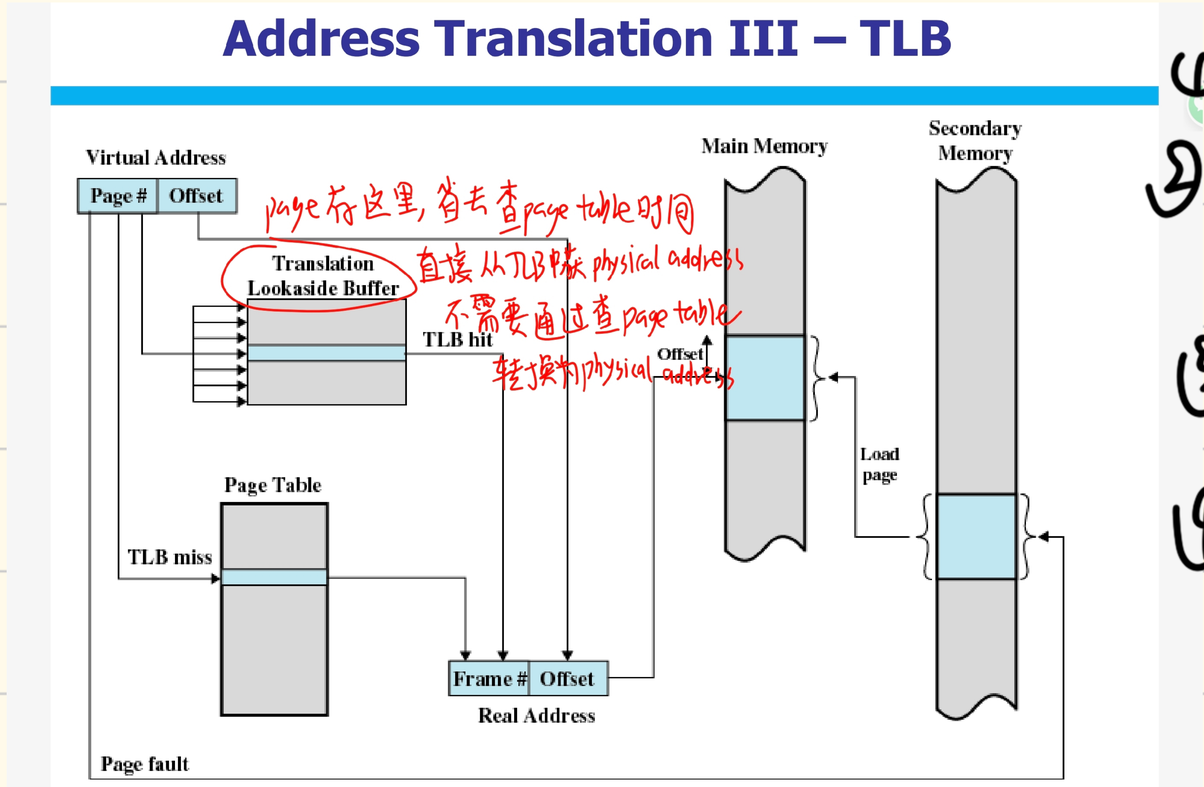
解决sparseness：

① 将1st level PTE设not valid可释放1024个pointer

② 少量无效1st level entries可save大量page table size

如何解决 memory access 慢(处理 address translation 的地址访问次数)

solution: Translation Lookaside Buffer (TLB) cache for page table translation



① CPU 内部的 HW cache

② 是 fully associative cache(找匹配项时对所有 entry 并行搜索)

③ 选择性保存 some identifier for process

④ context switch 时要 fresh TLB

↓

不同process可能用相同logical address但对应physical address不同

cache：在CPU与memory间，将经常访问的data放入高速缓存，CPU访问时先在cache中找(速度比从memory中找更快)

TLB
- TLB hit：TLB 命中
- TLB miss：TLB 失失要 access page table

↓

Locality 局部性
- temporal(time)：若 memory 中1个location被访问 则 same location 也很快被访问
- Spatial(space)：若 memory 中1个location被访问，则 location close by 也会被访问

	Offset			
	00	04	08	12
LPN=00				
LPN=01				
LPN=02		a[0]	a[1]	a[2]
LPN=03	a[3]	a[4]	a[5]	a[6]
LPN=04	a[7]	a[8]	a[9]	

Pattern of TLB hits/misses:

- Accessing a[0] causes a TLB miss
- Accessing a[1], a[2] will result in a TLB hit
- Accessing a[3] will cause TLB miss and again a[7]. All other accesses will cause TLB hit
- **TLB hit rate: (**number of TLB hits**) / (**total number of accesses**)** = 7/10=70%

访问a[0] TLB miss → a[0]一行放入TLB

访问a[1]、a[2] TLB hit

访问a[3] TLB miss → a[3]一行放入TLB

访问a[7] TLB miss → a[7]一行放入TLB

其他access均TLB hit

working set (WS): subset of pages being used (或 recently used)

page fault rate: 若 WS 不在 memory, rate ↑

Thrashing: all process 有 high page fault rate

Least recently used (LRU): 选 page 的方式 (最长时间未被用的 page 是最近不会用的)

↳ clock page replacement algorithm 时钟页面替换算法

week 4 concurrency

Threads

race condition

critical region

mutual exclusion

lock & synchronized method

deadlock

wait/notify

充分利用multi-core system 来提高 responsiveness

thread: a sequential execution stream within a process

thread间无protection + share resource

每个thread有个thread control block (TCB) → 有execution state、scheduling、pointer…

Multithreading: a single program made up of 多个不同concurrent execution threads

优: thread switch overhead low、thread creation/termination 更 fast

multithreaded process: PCB指向多个TCB PCB内切换thread (address space不变)

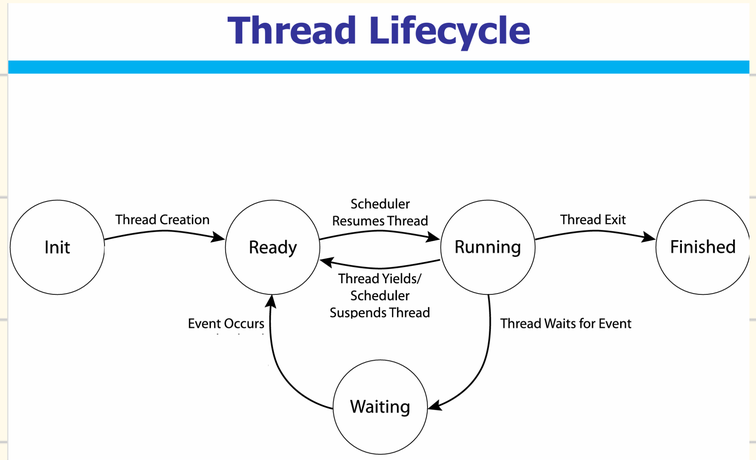
PCB间切换thread (address space 变)

process是资源分配的basic unit, thread是process中的execution unit共享资源

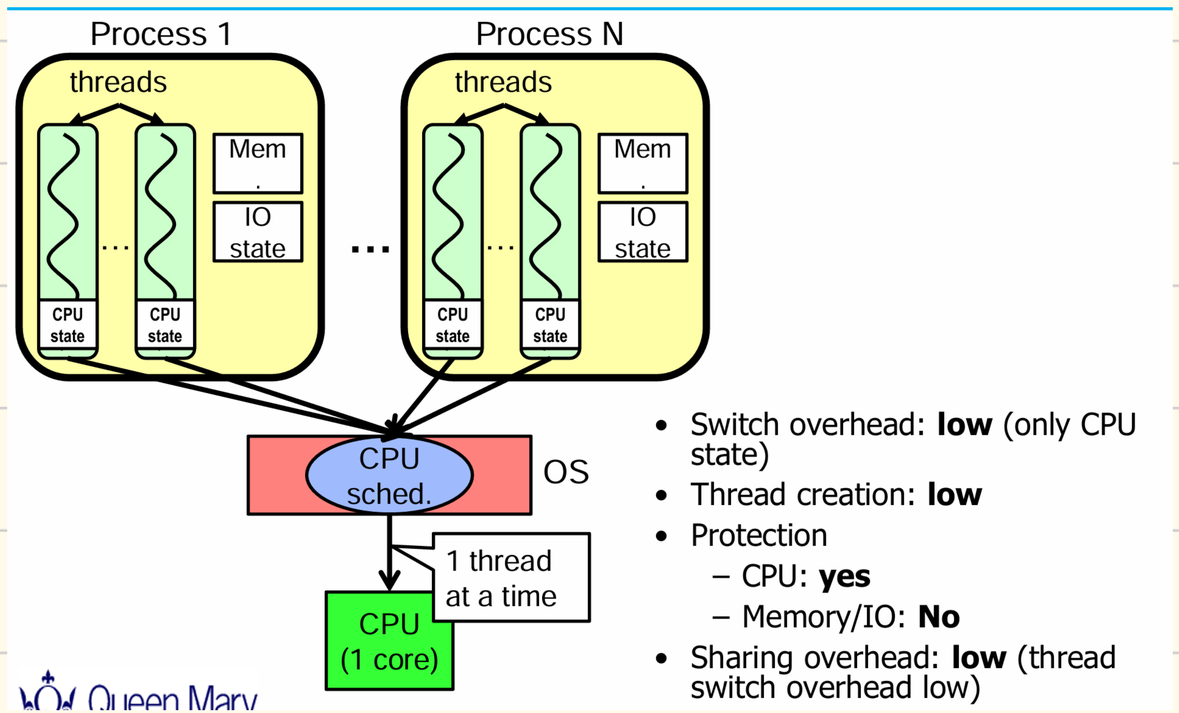
一个process中可多个thread同时执行(并发运行)可同时执行多个任务

process有独立address space → 实现内存保护,但thread间内存共享(并发)

run thread ① load state into CPU
② Load environment
③ jump to PC

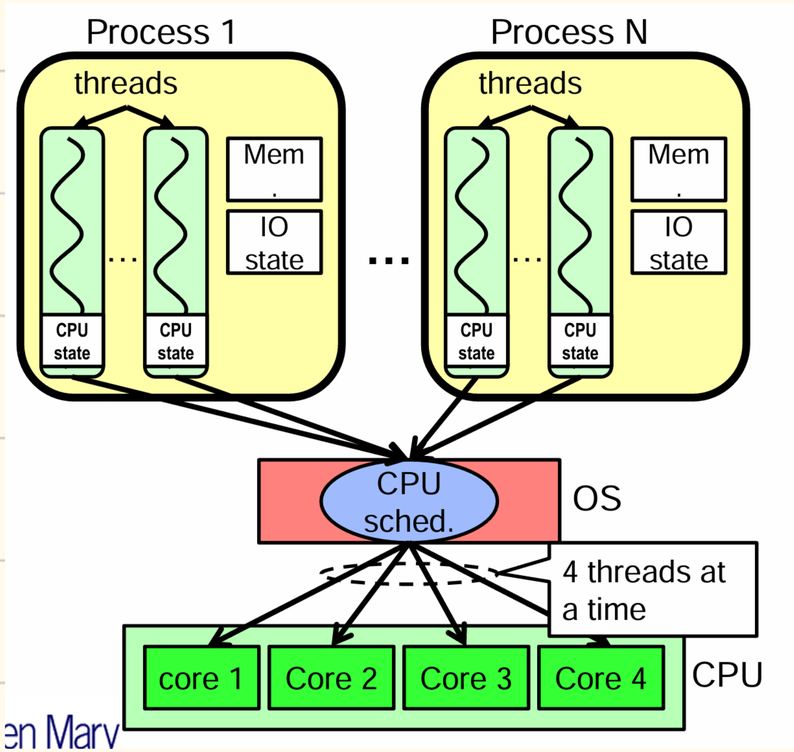


. thread on 1-core



- Switch overhead: **low** (only CPU state)
- Thread creation: **low**
- Protection
  - CPU: **yes**
  - Memory/IO: **No**
- Sharing overhead: **low** (thread switch overhead low)

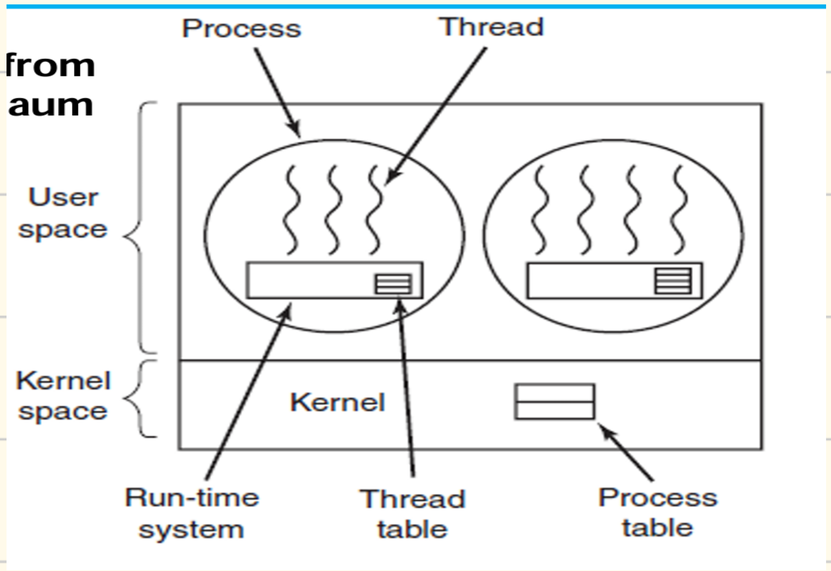
thread on multi-core



User level Threads (ULT)

cheap, faster, managed by program

不能handle multi-core processors



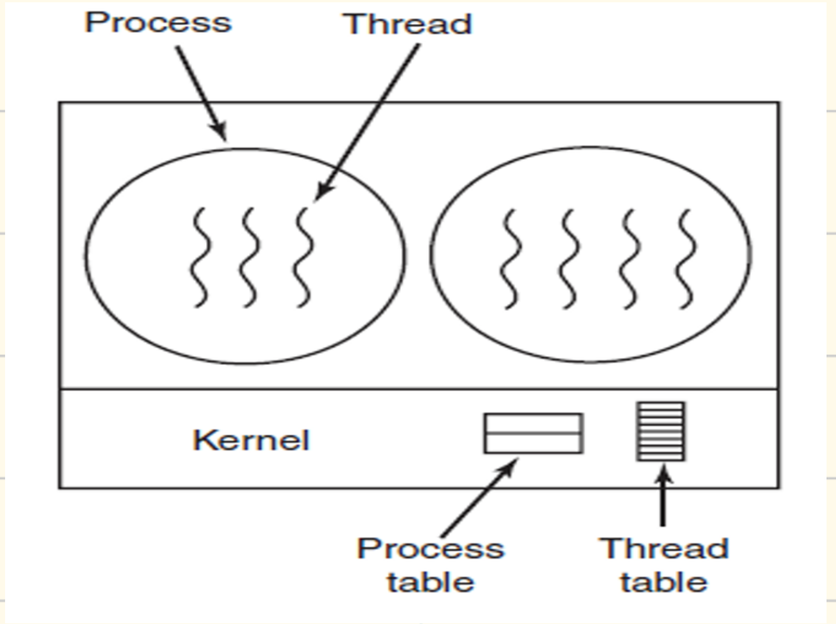
User space : all thread manage in user space
Run-time system有个Thread table 管理
1个process中all thread

Kernel space : 将process视为1个entity

Kernel level Threads (KLT)

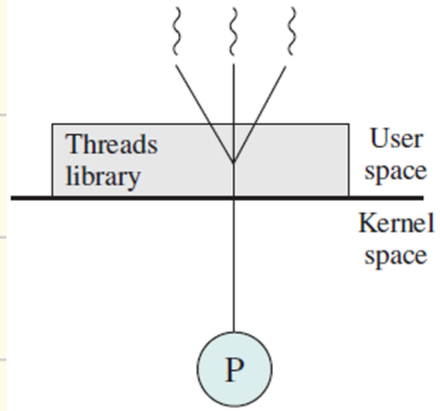
expensive, slower, manged by OS

能handle multi-core processors



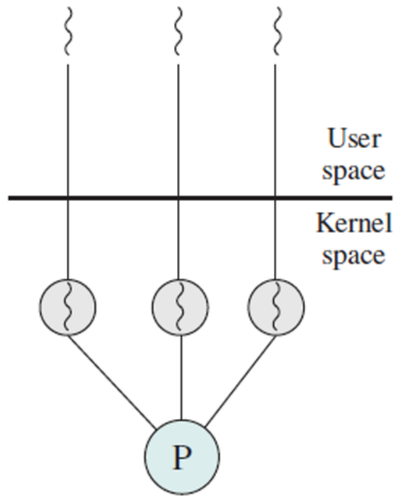
User space : 用户程序有多个thread但Kernel处理
内核管理Thread table

Kernel space : kernel有2个table



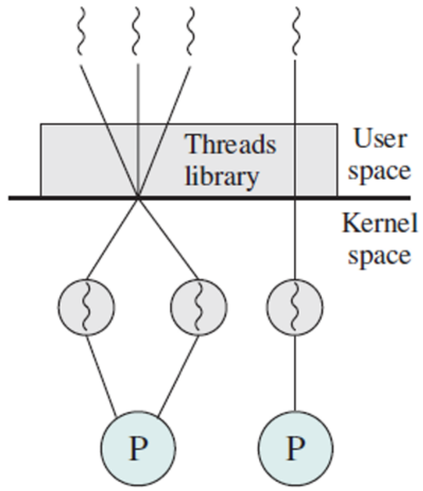
(a) Pure user–level

Kernel 调度1个 process



(b) Pure kernel–level

Kernel 调度 多个thread

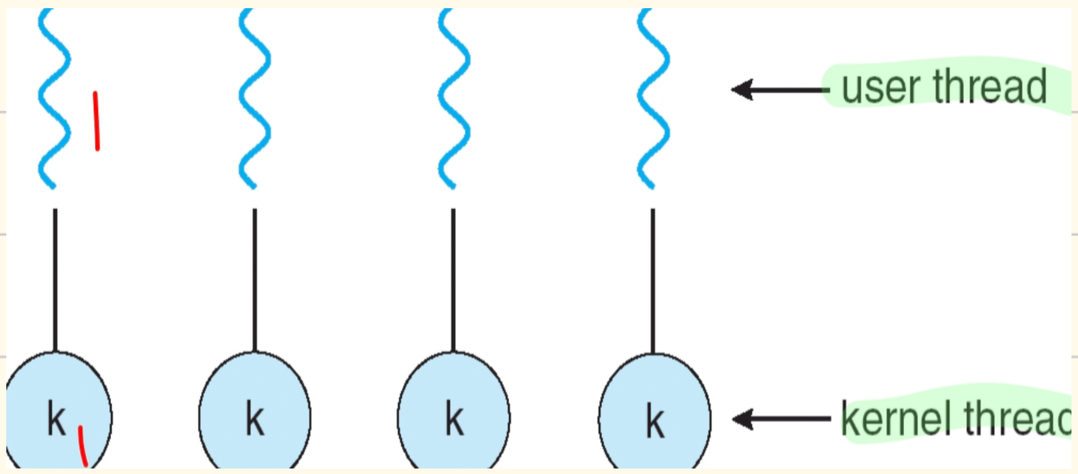


(c) Combined

多个ULT被少量KLT控制

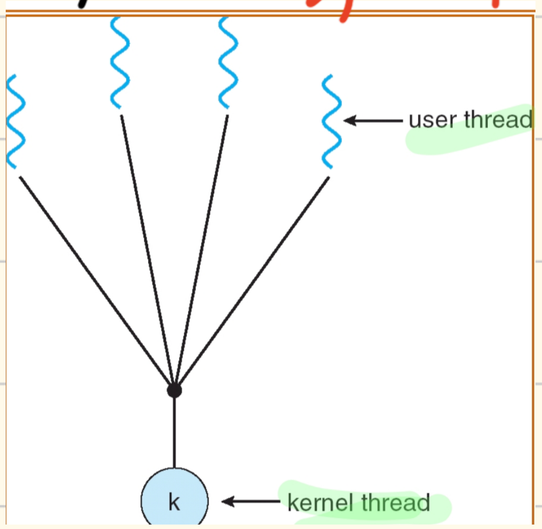
优：block system不会block 整个process
同程序多thread可多core并行处理

one-to-one thread Model



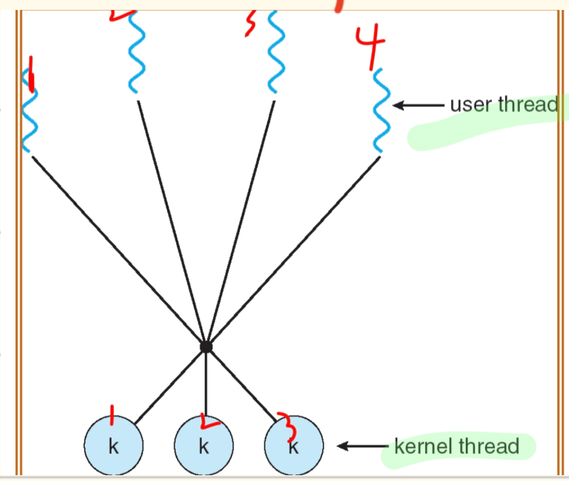
Many-to-one

1thread被block 全process被block



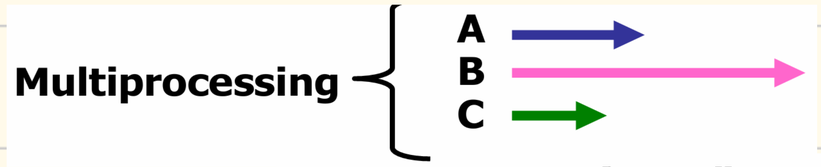
Many-to-Many

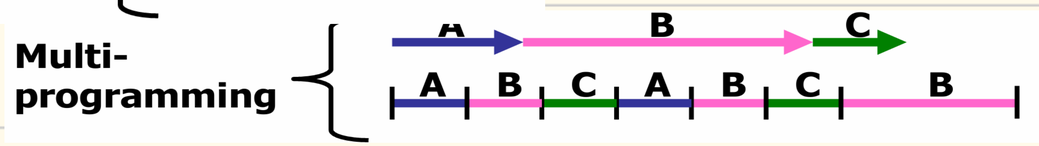
flexible
hard to implement



thread interleaving(交叉)

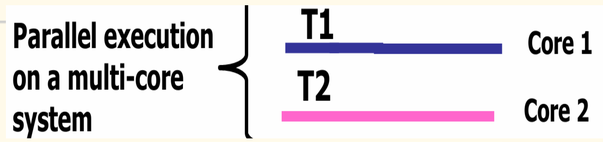
- multiprocessing : 多CPU
- multiprogramming : 多Job or process
- multithreading : 多 thread per process

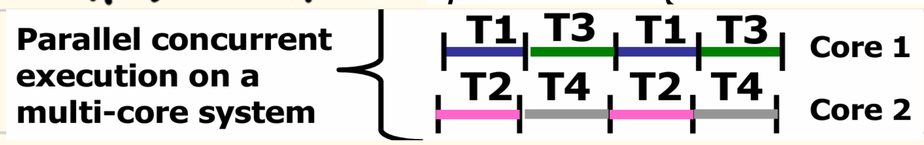




- concurrency : support more tasks by allowing all tasks to make a process
- parallelism : perform more tasks at the same time

concurrency + parallelism





atomic operation : operation that always runs to completion or not at all

race condition: a form of error in concurrent program

若thread对shared data的影响取决于调度thread的顺序 → occur behavior bewme unpredictable

how to avoid ↓

mutual exclusion: prevent 2 threads using a shared resource at the same time

1 thread 执行task时 exclude the other

critical region: part of the program over which mutual exclusion is needed

↳ requirement: 不允许2 threads 同时在 critical regions 中

no assumption 关于 speed 和 CPU数量(与之无关)

无thread在critical region外运行会block 其他threads

无 thread 一直wait for enter critical region

针对object 不是class

lock: 调用"synchronized"时 开始时take a lock, 结束时release it

thread 执行1个synchronized method时, 所有其他调用same object同步方法的thread会block

用synchronized "实现"

```
public synchronized int myMethod() {
  // code
}
```

```
public void addName(String name) {
    synchronized(this) {
        lastName = name;
        nameCount++;
    }
    nameList.add(name); // Runs without a lock
}
```

① 调用 synchronized 需 own the lock on 1 object instance

② 若lock已owned by other thread，该thread被block且is placed in entry queue

③ 当thread exit the synchronized lock会released

④ 若 entry queue 非空，JVM 任选1 thread 给lock

deadlock

Dinny philosophers 问题 (看看就行)

发生deadlock: 所有philosopher 同时eat(都拿left fork)

solution: 其中1个philosopher 先拿right fork

定义: a set of process (thread) is deadlocked if each process (thread) in set is waiting for an event that only another process (thread) in set can cause

4 necessary condition
① mutual exclusion: 资源不可共享
② hold and wait: wait 1个资源时 hold另1资源
③ no preemption 资源仅可 thread用完自动放弃
④ circuit wait: each thread hold 下一个thread 请求的resource

{ deadlock: process停止, 资源无法释放
starvation system整体使用资源有进步, 但特定program无法 get resource

thread termination：我们要了结前中止 thread

弃用方法：public final void destory() unsafe（放lock）

public final void suspend() deadlock（不放lock）

用↓

co-operative termination

- A thread can receive requests to terminate
- A thread must monitor such requests
- A thread must co-operate 通过release当前使用资源

用interrupt中止

Mailbox:

起初:

```
private T content;

public synchronized void write(T message) {
   content = message;
}
public synchronized T read() {
   return content;
}
```

用synchronized实现 mutual exclusion

问题: read first时, mailbox空 → read方法无法释放资源 → deadlock

解决: 若无while, thread可能wake up, 并发现通知的条件已改变(不满足thread运行条件)

```
private int content;
private boolean empty;

public synchronized int read() {
  while (empty) { wait(); }
  // mailbox now NOT empty

  empty = true;
  // mailbox now empty again
  notify();
  return content;
}
```

1. Variable – model the condition

2. Wait in a loop – while condition false

3. Update condition

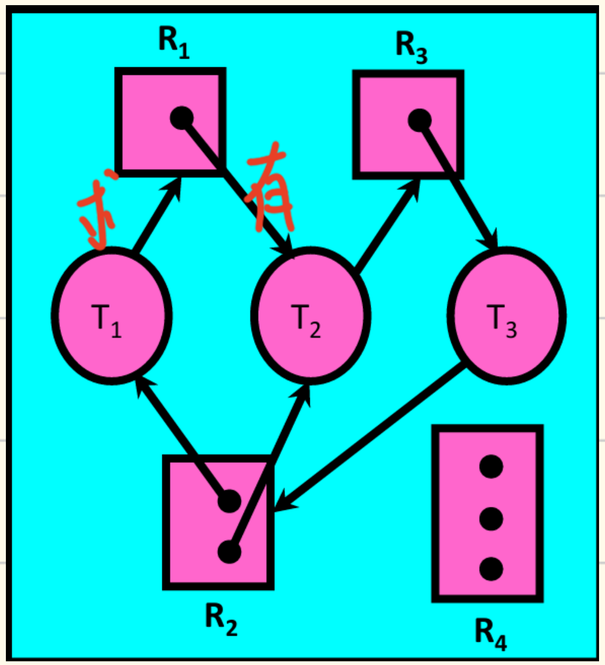
4. Notify other threads

```
private int content;
private boolean empty;

public synchronized void write(int message) {
  while (!empty) { wait(); }
  // mailbox now empty

  content = message;
  empty = false;  // mailbox now full again
  notify();
}
```

wait( ): cause current thread to wait 直到另1 thread调用notify( )

notify( )/notifyAll( ): wake up thread(1个/所有) that is waiting on monitor

Allocation graph with deadlock (有loop 就有deadlock)



也可用 $T_1 \to R_1 \to T_3 \to R_2 \to T_1$ (也是deadlock)

防止deadlock: dynamic avoidance by careful resource allocation

deadlock prevention

① break mutual exclusion

至少1个resource不可share —— 但一般不允许

② break hold and wait

ensure process 一次性request所有resource 并block该process直到所有request都允许

太低效：① 等待时间长 ② process不知要用的所有resource

③ 已分配给它的resource可能长时间未使用

③ break no preemption

try to take resource 一旦resource被分配 只适用于state易保存+恢复的

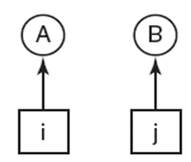
④ break circuit wait

define strict ordering of resource

- (a) Numerically ordered resources.
- (b) A resource graph.

1. Imagesetter
2. Scanner
3. Plotter
4. Tape drive
5. CD-ROM drive

(a)



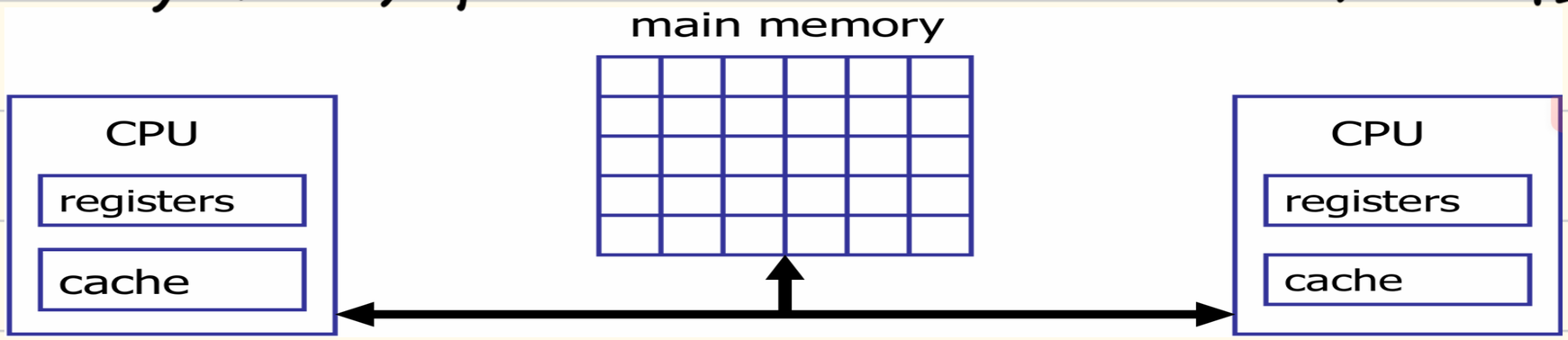
(b)

A process can request resource no. 2 and then resource no. 3, but not the other way around

How can a deadlock occur in this case? Consider i>j, i<j

Java memory model (shared memory multi-core system)

不能保证：
- interleaving：1 thread 更改影响另1 thread
- visibility of memory update：compiler 会延迟储存，对并发程序结果有影响



必须保证：
- atomic：synchronized block 自动执行（thread有lock时无其他thread执行）
- visibility：synchronized 内的更改必须在对thread间保持一致（否则别的thread还在用改前的旧值→error）
- ordering：order of synchronized block is guaranteed

ensure 1 thread 在同步块中写data，另1 thread在另1同步块 read data时更新1正确顺序显示。

happen-before guarantee：当1 operation 发生在另1 operation 前，first operation 保证排序在 second operation 前并对 second operation 可见

within a single thread rule：单个thread操作 are executed in the order the appear

lock and unlock rule：1 thread release lock 时保证之后有lock的thread看到first thread release lock 前的有更改

volatile fields rule：1 write to a volatile variable by 1 thread 发生在另1 thread read 前

Thread start rule：call start() 保证all action before start() 发生在new thread 任何 action 前

Thread join rule：all action in 1 thread 发生在另1 thread 成功从它的join()返回前

?

interrupt Disabling 屏蔽中断 —— 保证mutual exclusion

↓ process run until 它 invoke OS service or 被 interrupted

仅用于 ensure the safe checking and updating of lock status which thread running

Better Implementation of Locks by Disabling Interrupts

- The interrupt disabling is used only to ensure the safe checking and updating of the lock status (value) while the thread is running.

`int value = FREE;` (the lock variable)

```
Acquire() {
   disable interrupts;
   if (value == BUSY) {
      put thread on wait queue;
      Go to sleep();
      // Enable interrupts?
   } else {
      value = BUSY;
   }
   enable interrupts;
}
```

To protect the lock variable from race conditions, ensuring atomic operations when checking or updating it.

```
Release() {
   disable interrupts;
   if (anyone on wait queue) {
      take thread off wait queue
      Place on ready queue;
   } else {
      value = FREE;
   }
   enable interrupts;
}
```

Queen Mary University of London

binary semaphore 用于控制 shared data 的用的 synchronized tool

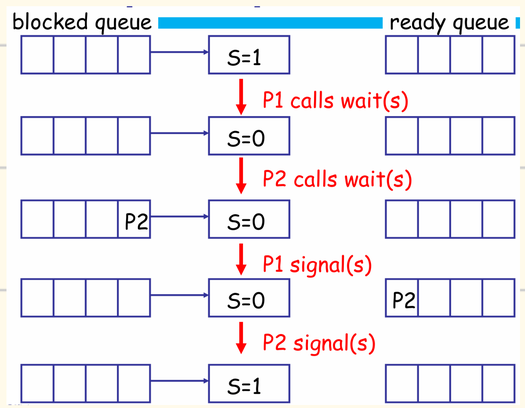
- Semaphore initialised to 1 or 0
- Two operations
  - **Wait** (P() – proberen in Dutch - test)
  - **Signal** (V() – verhogen in Dutch - increment)
- Wait
  - Blocks if semaphore = 0 → queue
  - If semaphore = 1, assigns 0 and proceeds
- Signal
  - Unblocks a queued process, if any
  - If no queued process, assigns 1

当前 resource 不可用，thread block

resource free 可用 → set 不可用

从 block queue 中取 1 个

block queue 为空 就 set 1. (free)



monitor: lock + 0~more condition variables,

用于 manage concurrent access to shared data

对比：semaphore 更 simple

monitor 更 structured